

用 TypeScript 與 AI 打造機器人

第二部：AI 決策篇 — 從規則邏輯到 AI 決策

建議時長：約 3 小時（含休息） / 須完成第一部後進行

一、本部分定位

第一部（機構篇）已帶領學生完成硬體控制與規則邏輯（if / else）驅動的反應式行為。第二部聚焦於本課程最核心的概念轉換：把『人類事先寫死的規則』換成『即時呼叫 AI 模型做決策』，讓學生實際體會 AI 作為機器人『決策層』的意義，並在過程中培養對 AI 產出內容保持驗證與批判的習慣。

本部分刻意把原先集中在單一 50 分鐘區塊的 AI 決策教學，拆解為三個子階段（6a、6b、6c），逐步疊加 async 呼叫、提示詞設計、結構化輸出解析、硬體整合等概念，避免多個新概念同時湧入造成學生無法排除錯誤來源。

二、學習目標

- 能說明『規則邏輯』與『AI 決策』兩種控制方式的差異與適用時機
- 能撰寫 TypeScript 程式呼叫 Claude API，並處理非同步（async/await）回應
- 能設計提示詞（prompt），使 AI 穩定回傳可解析的結構化 JSON
- 能將 AI 回傳的決策結果對應為實際馬達控制指令
- 能在人機協作情境中，閱讀、驗證並除錯 AI 產出的程式碼，而非照單全收

三、銜接第一部：概念回顧（10 分鐘）

建議以提問方式開場，喚回第一部結尾留下的思考：

- 『如果環境比原本設想的更複雜，寫死的規則還夠用嗎？』——帶出 AI 決策的動機
- 複習事件監聽（on）與非同步（async/await）的基本概念，因為第二部會大量使用
- 說明：AI 在這裡扮演的不是『幫你寫程式的工具』，而是『程式執行當下實際做判斷的角色』——這是兩種完全不同的 AI 使用方式，第二部前段（6a-6c）是後者，第 7 階段才會體驗前者

四、所需教具與軟體

第一部已完成之開發環境	Node.js、TypeScript、node-poweredup
LEGO Mindstorms Robot Inventor 套件	與第一部相同的硬體
Claude API 金鑰	本部分核心教具，須於課前申請並測試
穩定網路連線	AI API 呼叫需要網路，建議課前測試教室網路
文字編輯器／IDE	建議使用 VS Code 以獲得型別提示

△ *Claude API* 屬於本部分核心教具，務必於課前完成金鑰申請、額度確認，並在教室網路環境下實際測試過一次 *API* 呼叫，避免上課當天因網路限制或金鑰問題卡關。

五、時間表

回顧	第一部重點回顧與 AI 決策概念導入	10 分鐘	0:10
6a	AI API 呼叫（獨立練習，不接硬體）	20 分鐘	0:30
6b	提示詞設計討論（Prompt design）	15 分鐘	0:45
6c	將 AI 決策接上機器人	30 分鐘	1:15
休息	中場休息	10 分鐘	1:25
7	人機協作程式設計練習	40 分鐘	2:05
8	總結專題實作	45 分鐘	2:50
發表	成果發表與總結討論	20 分鐘	3:10

六、逐步教學內容

階段 6a：AI API 呼叫（獨立練習，不接硬體）（20 分鐘）

教學目標：在不涉及硬體的情況下，先讓學生單獨掌握『非同步呼叫 API → 取得回應 → 解析 JSON』這條技術鏈。刻意先移除藍牙與馬達的變數，讓學生除錯時只需面對程式碼本身，而不必同時排除硬體干擾。

```
async function askAI(distance: number, goal: string) {
  const response = await fetch("https://api.anthropic.com/v1/messages", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      model: "claude-sonnet-4-6",
      max_tokens: 200,
      messages: [{
        role: "user",
        content: `目前距離障礙物 ${distance} 公分，任務目標：${goal}。` +
          `請只回傳 JSON，格式為 {"action": "forward|stop|turn_left|turn_right", "power": 數字}`,
      }],
    }),
  });
  const data = await response.json();
  console.log(data.content[0].text); // 先印出原始回應，觀察格式
  return JSON.parse(data.content[0].text);
}
```

```
askAI(8, "避開障礙物並繼續前進");
```

練習方式：學生手動代入不同的 distance 數值（例如 3、8、50），觀察 AI 回傳的 action 是否合理變化，先在終端機（console）中驗證邏輯，尚未連接機器人。

💡 這個階段的重點不是『讓它動起來』，而是『看懂資料怎麼流動』。鼓勵學生先印出 data（完整回應物件）觀察它的結構，再逐步取出需要的欄位，養成先觀察資料形狀再寫解析邏輯的習慣。

階段 6b：提示詞設計討論（Prompt design）（15 分鐘）

教學目標：透過比較不同提示詞的效果，理解『提示詞怎麼寫』會直接影響 AI 輸出的穩定性與可用性，這是一堂討論課，不一定需要動手寫程式。

示範比較（可投影展示，讓學生預測哪一版本比較穩定）：

// 版本 A：模糊指示，容易夾雜說明文字
"機器人現在距離障礙物 8 公分，該怎麼辦？"

// 版本 B：明確要求格式，但沒有給範例
"距離 8 公分，請回傳 JSON 格式的動作指令。"

// 版本 C：明確格式 + 具體範例 + 限制欄位範圍
"距離 8 公分。請『只』回傳一行 JSON，不要任何說明文字，" +
"格式須為 {\\"action\\": \\"forward\\\"|\\\"stop\\\"|\\\"turn_left\\\"|\\\"turn_right\\\", \\"power\\": 0-100}"

- 討論：為什麼版本 C 通常比較穩定？『限制選項範圍』與『給出格式範例』分別解決了什麼問題？
- 討論：如果 AI 有時還是會夾雜文字說明，程式該怎麼處理？（帶出 6c 的錯誤處理）
- 延伸：可請學生分組嘗試改寫提示詞，比較實際回傳結果的穩定度

階段 6c：將 AI 決策接上機器人（30 分鐘）

教學目標：把 6a 已經驗證過的 AI 呼叫邏輯，正式整合進第一部的感測器 / 馬達控制迴路，完成『感測 → AI 決策 → 動作』的完整閉環。因為 AI 呼叫本身已在 6a 除錯過，此階段學生只需專注在『整合』本身，降低同時排除多重錯誤來源的難度。

```
hub.on("colorAndDistance", async (port, data) => {  
  try {  
    const decision = await askAI(data.distance, "避開障礙物並持續前進");  
    if (decision.action === "forward") {  
      await hub.setMotorSpeed("A", decision.power);  
    } else if (decision.action === "stop") {  
      await hub.setMotorSpeed("A", 0);  
    } else if (decision.action === "turn_left") {  
      await hub.setMotorSpeed("A", -decision.power);  
    }  
  } catch (err) {  
    console.error("AI 回應無法解析，機器人維持停止：", err);  
    await hub.setMotorSpeed("A", 0); // 解析失敗時的安全預設值  
  }  
});
```

△ 務必包含 `try/catch` 與『解析失敗時的安全預設動作』（例如停止）。AI 回應不一定每次都符合預期格式，若沒有錯誤處理，程式會直接崩潰，機器人甚至可能停在意外的動力狀態下，這也是向學生說明『為什麼 AI 決策系統需要安全防護』的好時機。

討論重點：跟第一部階段 5 的規則邏輯版本相比，這個版本的行為有什麼不同？AI 的判斷是否比較『靈活』，但也比較『不可預測』？兩者各自的優缺點是什麼？

階段 7：人機協作程式設計練習（40 分鐘）

教學目標：讓學生實際體驗『向 AI 提出需求、取得程式碼、在真實硬體上驗證與除錯』的協作流程——這裡的 AI 角色從『機器人的決策者』（6a-6c）切換為『幫忙寫程式的協作者』，刻意讓學生感受這兩種 AI 使用方式的差異。

- 學生以自然語言向 AI 助理（如 Claude）描述想要的行為，例如「幫我寫一段程式，讓機器人偵測到紅色就播放音效並後退」
- 取得 AI 產出的 TypeScript 程式碼後，在小組內先閱讀、預測執行結果，不要直接執行
- 實際部署到 Robot Inventor 上測試，記錄哪些部分符合預期、哪些需要修正
- 修正後重新提示 AI，形成「人 — AI — 硬體」的疊代迴路

💡 引導學生特別留意：AI 產出的程式碼是否使用了與 *node-poweredup* 實際 API 不符的方法名稱或參數？這種『看起來合理但實際不存在』的程式碼很常見，也是訓練學生查證、閱讀官方文件習慣的好機會。

階段 8：總結專題實作與發表（45 + 20 分鐘）

教學目標：整合前七個階段所學，完成一個結合感測器、馬達與 AI 決策的完整機器人行為，並上台展示。建議把原先『60 分鐘同時做完加發表』拆為『45 分鐘實作』+『20 分鐘發表與總結討論』，讓實作與發表不互相擠壓。

專題方向範例（可依教室資源挑選 1-2 項，或讓學生自選）：

- 機器人會口語化描述目前感測到的環境（例如經文字轉語音）
- 機器人向 AI 詢問走出簡易迷宮的路徑規劃，再逐步執行
- 機器人依據使用者輸入的目標，由 AI 動態調整巡邏或避障策略

💡 若團隊時間不足以完成『AI 決策 + 完整場景』，可接受降階版本：例如只用固定的兩三種情境測試 AI 決策是否合理，而非要求即時應對任意環境——專題的重點是體現整合能力與驗證過程，不是場景的複雜度。

七、第二部評量重點

- 是否理解『規則邏輯』與『AI 決策』的差異，並能舉例說明各自適用情境
- AI API 呼叫程式是否具備基本錯誤處理（`try/catch`、預設安全動作）
- 提示詞設計是否有考慮到格式穩定性（明確格式要求、限制選項範圍）
- 在人機協作練習中，是否實際驗證並修正了 AI 產出的程式碼，而非直接照抄執行
- 總結專題是否完整呈現『感測 → AI 決策 → 動作』的閉環，並能在發表時說明遇到的

問題與排除過程

八、教師提醒

- 務必提醒學生：AI 的建議或程式碼永遠需要在硬體上實際驗證，不能假設它一定正確
- API 呼叫有網路延遲，機器人反應會比第一部的規則邏輯版本明顯慢，這是正常現象，可藉此討論『即時性』與『決策品質』之間的取捨
- 若教室網路不穩定或 API 額度有限，可讓部分小組共用金鑰、錯開呼叫時間，或安排部分練習以預先錄製的 API 回應（mock data）進行
- 階段 7 容易出現 AI 產出程式碼『看似合理但實際跑不動』的情況，這是刻意保留的教學情境，不需要教師提前排除，反而是討論『驗證 AI 產出』的最佳時機